학습 교안

하네스 엔지니어링 입문 2단계

AI 에이전트와 함께 규칙, 기준, 변경, 되돌리기를 익힌다 도서 대출 목록 프로그램 하나

OpenCode · OpenRouter · Python · pytest · Git

준비하기 + 1~7장 · 1단계 이후의 초급 실습

강사 김대철

작성일 2026.11.12

실습 환경 Windows + Python + Git + OpenCode

실습 저장소 https://github.com/sinaboro/harness-beginner-tutorial

교안의 명령과 테스트 결과는 Windows PowerShell 에서 실제로 실행해 얻은 것입니다.
AI 의 답변은 실행할 때마다 달라지므로, 교안에는 답변 대신 확인 방법을 실었습니다.

목차

과정 안내 . . . 5

단계 안내 . . . 5

2단계 교시표 . . . 5

도커로 실행하기 (선택) . . . 5

0단계 교시표 . . . 6

1단계 교시표 . . . 6

저장소 구조 . . . 7

시작하기 . . . 7

원칙 세 가지 . . . 7

강사 안내 . . . 8

2단계 준비하기 (도커) . . . 9

1. 컨테이너 켜기 (Windows) . . . 9

2. 도구 확인 (컨테이너 안) . . . 9

3. 작업 폴더 만들기 (컨테이너 안) . . . 9

4. 1단계 도구 복사하기 (컨테이너 안) . . . 9

확인 체크리스트 . . . 10

2단계 1장. 도서관 요구사항 쓰기 . . . 11

학습 목표 . . . 11

1. 개념 — 규칙은 상태에서 나온다 . . . 11

2. 따라 하기 . . . 11

3. 확인하기 . . . 12

4. 자주 만나는 오류 . . . 12

5. 확인 문제 . . . 12

2단계 2장. 책 목록을 파일에 저장하기 . . . 13

학습 목표 . . . 13

1. 개념 — 상태를 파일에 남긴다 . . . 13

2. 따라 하기 . . . 13

3. 확인하기 . . . 15

4. 자주 만나는 오류 . . . 15

5. 확인 문제 . . . 15

2단계 3장. 대출과 반납: 여러 기능을 나눠 맡기기 . . . 16

학습 목표 . . . 16

1. 개념 — 한 번에 맡기지 않는다 . . . 16

2. 따라 하기 . . . 16

3. 확인하기 . . . 18

4. 자주 만나는 오류 . . . 18
5. 확인 문제 . . . 18
2단계 4장. AI 가 만든 실수 찾기 . . . 19
학습 목표 . . . 19
1. 개념 — 테스트는 약속만 확인한다 . . . 19
2. 따라 하기 . . . 19
3. 확인하기 . . . 21
4. 자주 만나는 오류 . . . 22
5. 확인 문제 . . . 22
2단계 5장. 요구사항이 바뀌었을 때 . . . 23
학습 목표 . . . 23
1. 개념 — 바뀔 때는 기준부터 . . . 23
2. 따라 하기 . . . 23
3. 확인하기 . . . 25
4. 자주 만나는 오류 . . . 25
5. 확인 문제 . . . 25
2단계 6장. 잘못된 변경을 되돌리기 . . . 26
학습 목표 . . . 26
1. 개념 — 되돌리는 방법은 두 가지다 . . . 26
2. 따라 하기 . . . 26
3. 확인하기 . . . 28
4. 자주 만나는 오류 . . . 28
5. 확인 문제 . . . 29
2단계 7장. 미니 프로젝트: 도서 대출 목록 완성 . . . 30
학습 목표 . . . 30
1. 개념 — 지금까지의 고리 . . . 30
2. 따라 하기 . . . 30
3. 확인하기 . . . 32
4. 막혔을 때 . . . 33
5. 확인 문제 . . . 33
다음 단계 . . . 33
부록. 템플릿과 예제 . . . 34
A-1. 도서 대출 요구사항 최종본 (7장) . . . 34
A-2. 요구사항 변경 요청서 (5장) . . . 35
A-3. 책 저장 작업 지시서 (2장) . . . 35
A-4. 대출과 반납 작업 지시서 (3장) . . . 36
A-5. 미니 프로젝트 작업 지시서 (7장) . . . 36
A-6. 책 등록과 저장 테스트 (2장) . . . 37

A-7. 대출과 반납 테스트 1차 (3장) 38
A-8. 대출과 반납 테스트 2차 (5장) 39
A-9. 4장 연습용 초안 (검증 전) 41

과정 안내

AI 에이전트에게 개발을 맡기되, 규칙·권한·검증으로 통제하고, 통과한 것만 저장하는 법을 단계별로 배웁니다.

코드를 먼저 배운 뒤에 AI 를 쓰는 과정이 아닙니다. AI 에게 맡기면서 코드와 개발 원리를 함께 이해하는 과정입니다. 그래서 각 단계는 "하네스의 한 요소"를 하나씩 익히도록 설계했습니다.

대상	프로그래밍을 처음 배우는 사람 (코딩 경험이 없어도 됩니다)
준비물	Windows PC, Git, OpenCode, OpenRouter 계정 (1단계부터 Python 추가)
필요 없는 것	Docker, 데이터베이스, 웹 서버 (0·1단계 기준)
에이전트	OpenCode (OpenRouter 모델 사용)

단계 안내

단계	대상	내용	상태
0단계	코드를 전혀 모르는 사람	프로그래밍 언어 없이, HTML·CSS 명함 페이지 한 장을 AI 에게 맡기고 하네스의 원리를 익힌다	**이 저장소에 있음** (`docs/stage0/`)
1단계	생초보	Python 계산기와 할 일 목록 프로그램으로, 테스트와 저장 장치까지 익힌다	**이 저장소에 있음** (`docs/stage1/`)
2단계	초급	도서 대출 목록 프로그램으로, 저장, 상태 규칙, 요구사항 변경, 되돌리기를 익힌다	**이 저장소에 있음** (`docs/stage2/`)
3단계	실무형	3일 과정. FastAPI, PostgreSQL, Docker 까지 연결하는 전체 실습	별도 저장소 harness-crud-lab

2단계 교시표

장	주제	하네스 개념	결과물
준비하기	1단계 도구 확인, 작업 폴더 만들기	도구와 실행 환경	설치 확인
1장	도서관 요구사항 쓰기	생활 규칙을 상태로 바꾸기	도서관 요구사항
2장	파일에 저장하기	상태를 파일로 남기기	책 등록, JSON 저장
3장	대출과 반납	작은 지시로 나눠 맡기기, 상태 규칙	대출과 반납 기능
4장	AI 가 만든 실수 찾기	테스트가 잡는 실수와 못 잡는 실수	빠진 약속을 추가한 테스트
5장	요구사항이 바뀌었을 때	기준은 사람이, 구현은 AI 가	변경 요청서, 대출 기간 기록
6장	잘못된 변경을 되돌리기	`restore` 와 `revert`	되돌린 기록
7장	미니 프로젝트: 도서 대출 목록	1~6장 종합	완성된 프로그램, 과정 기록

도커로 실행하기 (선택)

PC 에 Python 과 OpenCode 를 직접 설치하지 않고, 컨테이너 안에서 0·1·2단계를 할 수 있습니다. 방법은 `docs/docker/00_도커로_실행하기.md` 를 봅니다. 편집은 Windows 의 VS Code 에서, 명령은 컨테이너 안에서 합니다.

0단계 교시표

장	주제	결과물
0장	준비하기 (Git, OpenCode, OpenRouter 연결)	설치 확인
1장	AI 와 하네스 체험	조건 없이 시킨 것과 조건을 붙인 것의 비교
2장	말로 된 요구사항 쓰기	명함 요구사항
3장	작업 지시서 쓰기	여섯 칸 지시서
4장	규칙과 권한 세우기	`AGENTS.md`, 권한 설정 확인
5장	체크리스트로 확인하고 저장하기	체크리스트 확인 후 커밋
6장	미니 프로젝트: 자기소개 명함 페이지	HTML·CSS 명함 페이지, 과정 기록

1단계 교시표

교시	주제	배우는 것	결과물
0장	준비하기	Python, Git, OpenCode, OpenRouter 연결	설치 확인 체크리스트
1장	하네스란 무엇인가	같은 부탁을 조건 없이 한 것과 조건을 붙인 것의 차이	비교 기록
2장	요구사항 쓰기	"이렇게 하면 → 이런 결과"로 흐리지 않게 쓰기	계산기 요구사항
3장	작업 지시서 쓰기	목표·범위·하지 말 것·완료 조건 여섯 칸	지시서와 계획 검토
4장	규칙과 권한 세우기	`AGENTS.md`(부탁)와 `opencode.jsonc`(잠금장치)	규칙 문서, 권한 확인
5장	검증하고 저장하기	빨강→초록, 검증을 통과해야 커밋되는 훅	통과한 커밋
6장	미니 프로젝트	할 일 목록을 AI 에게 맡기고 통제하기	할 일 목록 프로그램, 과정 기록

각 장은 **학습 목표 → 개념 → 따라 하기 → 확인하기 → 자주 만나는 오류 → 확인 문제** 순서입니다.

저장소 구조

```
harness-beginner-tutorial/
├ README.md                 이 문서
├ docs/
|   ├ stage0/               0단계 교안 (0장~6장)
|   ├ stage1/               1단계 교안 (0장~6장)
|   ├ docker/               도커로 실행하기 (0·1·2단계 공통)
|   ├ stage2/               2단계 교안 (준비하기, 1장~7장)
|   ├ 하네스_입문_0단계_교안.pdf
|   ├ 하네스_입문_1단계_교안.pdf
|   └ 하네스_입문_2단계_교안.pdf
├ templates/                수업에서 복사해서 쓰는 틀과 파일
|   ├ stage0/               0단계 틀 (명함 요구사항·지시서·규칙·권한·체크리스트)
|   ├ stage2/               2단계 틀 (도서관 요구사항·지시서·변경 요청서·테스트 기준)
|   ├ REQ_빈칸.md            2장: 요구사항 빈칸 틀
|   ├ TASK_빈칸.md           3장: 작업 지시서 빈칸 틀
|   ├ AGENTS_빈칸.md         4장: 규칙 문서 빈칸 틀
|   ├ opencode.jsonc        4장: 권한 설정 (완성본)
|   ├ REQUIREMENTS.md       6장: 할 일 목록 요구사항
|   ├ TASK_todo.md          6장: 할 일 목록 작업 지시서
|   ├ tests/test_todo.py    6장: 할 일 목록 테스트 (AI 가 고칠 수 없는 기준)
|   ├ verify.py             5장: 검증 스크립트
|   ├ pre-commit            5장: 커밋 전 검증 훅
|   └ .gitattributes        줄바꿈 설정
├ docker/                   도커 실행 환경 (0·1·2단계 공용)
|   ├ Dockerfile            Python, pytest, Git, OpenCode 가 든 이미지 설계도
|   ├ docker-compose.yml    작업 폴더와 템플릿 폴더를 연결하는 설정
|   └ .env.example          경로와 Git 이름 설정 견본
├ examples/
|   ├ calc/                 1단계 4·5장에서 쓰는 계산기 예제 (multiply 가 빠진 상태)
|   └ stage2/               2단계 4장에서 쓰는 연습용 초안 (검증 전, 실수 포함)
└ tools/
    └ build_guide.py        교안 PDF 를 만드는 스크립트 (인자: 0 또는 1)
```

시작하기

0단계(`docs/stage0/`), 1단계(`docs/stage1/`), 2단계(`docs/stage2/`) 순서로 따라 합니다.

■ **[PS] 윈도우 PowerShell** 저장소를 내려받습니다.

```
git clone https://github.com/sinaboro/harness-beginner-tutorial C:\harness-beginner-tutorial
```

교안은 PDF 로도 볼 수 있습니다. `docs/하네스_입문_0단계_교안.pdf`, `docs/하네스_입문_1단계_교안.pdf`, `docs/하네스_입문_2단계_교안.pdf` 를 여세요.

0단계부터 순서대로 합니다. 0단계는 Python 이 필요 없고, 1단계부터 Python 을 씁니다.

원칙 세 가지

1. **AI 가 쓰고, 사람이 판단합니다.** 코드는 AI 가 쓰더라도, 계획을 승인하고 결과를 읽고 저장하는 판단은 사람이 합니다.
2. **기준은 AI 가 고치지 않습니다.** 테스트와 검증 스크립트, 규칙 문서는 권한 설정으로 AI 가 고치지 못하게 막습니다.
3. **통과한 것만 저장합니다.** 검증이 실패하면 커밋 자체가 취소됩니다.

강사 안내

- 교안의 실행 결과는 0장~6장의 명령을 실제 Windows PowerShell 에서 확인한 것을 기준으로 합니다.
  OpenCode 의 AI 답변은 실행할 때마다 다르므로, 교안에는 답변 내용 대신 **확인 방법**을 적었습니다.
- 6장에는 정답 코드를 싣지 않았습니다. 학생이 테스트와 지시서를 읽고 AI 와 대화하며 맞춰 가도록 한 것입니다.
- 3단계(3일 과정)는 별도 저장소 harness-crud-lab 에 있습니다. 2단계 7장을 마치면 그 저장소로 이어 갑니다.

2단계 준비하기 (도커)

시간	30분
준비물	1단계를 마친 상태 (도커 이미지와 OpenRouter 연결이 있다)
결과	컨테이너 안에 도서 대출 목록 작업 폴더가 만들어지고, 1단계 도구가 그대로 동작한다

2단계는 1단계와 **같은 도커 이미지**를 씁니다. 새로 설치할 것은 없습니다.

> **역할 나누기**
> - **Windows 쪽(PowerShell)**: 컨테이너를 켠다. `[Windows]` 표시.
> - **컨테이너 쪽(bash)**: 작업 폴더를 만들고 실습한다. `[컨테이너]` 표시.

1. 컨테이너 켜기 (Windows)

[Windows] (PowerShell)

```
cd C:\harness-beginner-tutorial\docker
docker compose run --rm workspace
```

2. 도구 확인 (컨테이너 안)

[컨테이너]

```
python --version
python -m pytest --version
git --version
opencode --version
```

네 줄 모두 버전이 나와야 합니다. OpenCode 연결은 `opencode` → `/models` 로 확인하고 `/exit` 로 나옵니다.

3. 작업 폴더 만들기 (컨테이너 안)

[컨테이너]

```
mkdir -p /workspace/library/tests
cd /workspace/library
git init
git config core.hooksPath .githooks
```

> 이 폴더 `/workspace/library` 는 Windows 의 `C:\harness-lab\library` 입니다. 2장부터 7장까지 모두 여기서 합니다.

4. 1단계 도구 복사하기 (컨테이너 안)

[컨테이너]

```
T="/tutorial/templates"
cp "$T/verify.py" .
cp "/tutorial/templates/stage2/pytest.ini" .
cp "$T/opencode.jsonc" .
cp "$T/AGENTS_빈칸.md" AGENTS.md
cp "$T/.gitattributes" .
mkdir -p .githooks
```

```
cp "$T/pre-commit" .githooks/pre-commit
echo ".env" > .gitignore
ls -la
```

- `AGENTS.md` 는 1단계 4장에서 채운 것이 있다면 그것을 옮겨 씁니다. 없다면 빈칸 틀을 4장과 같은 방식으로 채웁니다.

확인 체크리스트

- [ ] 도구 네 가지 버전이 나온다
- [ ] `/workspace/library` 에 `verify.py`, `pytest.ini`, `opencode.jsonc`, `AGENTS.md`, `.githooks` 가 있다
- [ ] `git status` 가 실행된다

2단계 1장. 도서관 요구사항 쓰기

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	준비하기 완료
결과	도서 대출 목록의 요구사항을 "이렇게 하면 → 이런 결과 → 잘못 쓰면"으로 쓴다

학습 목표

1. 생활 속 규칙("이미 빌려 간 책은 못 빌린다")을 요구사항으로 옮긴다.
2. 상태(대출 가능, 대출 중)를 명확한 말로 정한다.
3. "잘못 쓰면" 칸을 채워 빈틈을 찾는다.

1. 개념 — 규칙은 상태에서 나온다

도서관에는 책 한 권마다 **상태**가 있습니다. 대출 가능, 대출 중입니다. 대부분의 규칙은 상태에서 나옵니다.

생활 속 규칙	상태로 바꾼 말	요구사항
빌려 간 책은 다른 사람이 못 빌린다	대출 중이면 대출할 수 없다	대출 중인 책을 빌리려 하면 오류
빌리지 않은 책은 반납할 수 없다	대출 가능이면 반납할 수 없다	대출 가능한 책을 반납하려 하면 오류
빌리는 사람 이름이 있어야 한다	이름이 비면 대출할 수 없다	이름이 공백이면 오류

> 1단계 2장의 "잘못 쓰면" 칸이 여기서 가장 중요합니다. 규칙마다 **잘못된 경우**가 하나씩 있습니다.

2. 따라 하기

1단계. 생활 규칙 찾기 (15분)

도서관에 가면 볼 수 있는 규칙을 다섯 개 적습니다. 예: "책은 한 번에 한 권만 빌린다", "반납 기한이 있다".

생활 속 규칙	이 프로그램에서 다룰까? (예/아니오)	아니오면 이유

2단계. 요구사항 틀 채우기 (40분)

1단계 틀 `templates\REQ_빈칸.md` 를 복사합니다.

[컨테이너]

```
mkdir -p /workspace/library-req
cp /tutorial/templates/REQ_빈칸.md /workspace/library-req/요구사항.md
# (Windows) /workspace/library-req/요구사항.md 을(를) VS Code 로 열어 고칩니다
```

다음 기능을 채웁니다.

- **R1. 책 등록** (제목, 지은이)
- **R2. 목록 보기** (전체, 대출 가능, 대출 중)

- **R3. 대출** (번호와 빌리는 사람 이름)
- **R4. 반납** (번호)
- **R5. 파일 저장과 읽기**

R3, R4 에는 반드시 "잘못 쓰면" 칸을 채웁니다.

- R3: 없는 번호, 이미 대출 중, 이름이 공백
- R4: 없는 번호, 대출 중이 아닌 책

3단계. 빈칸 점검하기 (20분)

아래 질문으로 점검합니다.

- ■ "책 목록" 같은 흐린 말이 있나? 어떤 목록인지 상태로 말했나?
- ■ 반납하면 빌린 사람 이름은 어떻게 되나? 요구사항에 적혀 있나?
- ■ 책 번호는 어떻게 정해지나? 지우는 기능이 있으면 번호는 어떻게 되나?
- ■ "만들지 않는 것"을 적었나? (연체, 날짜 계산 등)

> 세 번째 질문이 함정입니다. 4교시와 5교시에서 이 빈틈이 다시 나옵니다.

4단계. 옆 사람과 바꿔 읽기 (25분)

옆 사람의 요구사항에서 **"반납하면 빌린 사람 이름은 어떻게 되나요?"** 같은 질문을 하나씩 찾아 적습니다. 대답이 없으면 그 요구사항은 빈틈이 있는 것입니다.

3. 확인하기

- ■ 상태(대출 가능, 대출 중)가 요구사항에 명확히 적혀 있다.
- ■ R3, R4 의 "잘못 쓰면" 칸이 모두 채워져 있다.
- ■ 반납 시 빌린 사람 이름이 어떻게 되는지 요구사항에 적혀 있다.

4. 자주 만나는 오류

증상	해결
"관리를 잘한다" 같은 흐린 말이 나온다	상태로 바꾼다. 예: "대출 중인 책 목록을 보여 준다"
반납 후 상태를 빠뜨렸다	반납 전과 후의 상태를 한 줄씩 적는다.

5. 확인 문제

1. 대출 중인 책을 다시 빌리려 하면 어떤 결과가 나와야 하나? 요구사항에서 찾아 적으시오.
2. 반납하면 `borrower`(빌린 사람)는 무엇이 되어야 하나?
3. 요구사항에 "연체"를 넣지 않은 이유를 "만들지 않는 것"과 연결해서 설명하시오.

2단계 2장. 책 목록을 파일에 저장하기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	1장 요구사항, 준비하기 완료
결과	책 등록과 목록, JSON 파일 저장을 AI 가 만들고, 테스트로 확인해 저장한다

학습 목표

1. 프로그램이 꺼지면 사라지는 것과 파일에 남는 것의 차이를 설명한다.
2. JSON 이라는 파일 형식을 읽을 수 있다.
3. 기준(테스트)을 먼저 보고, 빨강에서 초록으로 가는 흐름을 따른다.

1. 개념 — 상태를 파일에 남긴다

1단계 할 일 목록은 메모리에만 있었습니다. 프로그램을 끄면 사라졌습니다. 도서관은 **책 목록이 꺼져도 남아야** 합니다.

JSON 은 데이터를 글로 적는 형식입니다. 파이썬의 딕셔너리와 거의 같은 모양입니다.

```
[
  {"id": 1, "title": "데미안", "author": "헤세", "status": "available", "borrower": null}
]
```

JSON	파이썬
`{ }`	딕셔너리 `{}`
`[ ]`	리스트 `[]`
`null`	`None`
`true` / `false`	`True` / `False`

2. 따라 하기

1단계. 기준(테스트) 읽기 (15분)

[컨테이너]

```
cd /workspace/library
cp /tutorial/templates/stage2/tests/test_books.py tests/
# (Windows) tests\test_books.py 을(를) VS Code 로 열어 고칩니다
```

아래 세 질문에 답합니다.

1. `add_book` 이 돌려주는 책은 어떤 모양인가? (딕셔너리의 키 다섯 개)
2. 공백뿐인 제목은 어떻게 되어야 하나?
3. 저장 파일이 아직 없으면 `load_books` 는 무엇을 돌려주나?

> 이 세 답이 AI 가 지켜야 할 약속입니다.

2단계. 빨강 확인하기 (10분)

[컨테이너]

```
python -m pytest tests/test_books.py
```

`library.py` 가 없으므로 실패합니다. 이것이 빨강입니다. 오류 줄에서 `No module named 'library'` 또는 비슷한 문장을 찾아 적습니다.

3단계. AI 에게 만들게 하기 (40분)

[컨테이너]

```
cp /tutorial/templates/stage2/TASK_책저장.md .
opencode
```

■ [AI] OpenCode 입력 칸

```
TASK_책저장.md 를 읽고 그 지시대로 작업해 줘. 먼저 계획만 말하고 내 승인을 기다려.
```

계획을 검토합니다.

- ■ 고칠 파일이 `library.py` 하나뿐인가?
- ■ `lend_book`, `return_book` 을 만들겠다는 말이 없는가?
- ■ `tests/` 를 고치겠다는 말이 없는가?

맞으면 `진행` 이라고 답합니다. 끝나면 `/exit` 로 나옵니다.

4단계. 빨강에서 초록으로 (15분)

[컨테이너]

```
python -m pytest tests/test_books.py
```

모두 통과해야 합니다. 실패가 있다면 실패한 테스트 이름과 `assert` 줄을 AI 에게 붙여 주고, 원인부터 설명하게 합니다. 테스트를 고쳐서 통과시키라고 하면 안 됩니다.

[컨테이너]

```
diff tests/test_books.py /tutorial/templates/stage2/tests/test_books.py
```

아무것도 나오지 않으면 테스트는 바뀌지 않은 것입니다.

5단계. 저장 파일 눈으로 보기 (10분)

파이썬으로 책을 두 권 넣고 저장해 봅니다.

[컨테이너]

```
python -c "from library import add_book, save_books; b=[]; add_book(b,'데미안','헤세'); add_book(b,'노인과 바다','헤밍웨이'); save_books
  (b,'books.json'); print('저장 완료')"
# (Windows) books.json 을(를) VS Code 로 열어 고칩니다
```

`books.json` 을 열어 보고, 1장의 요구사항 R5 와 대조합니다. 한글이 깨지지 않았는지 확인합니다.

> 프로그램을 끄고 다시 켜도 `books.json` 은 남습니다. 이것이 저장입니다.

6단계. 검증하고 저장하기 (10분)

먼저 책 데이터 파일을 저장소에서 제외합니다. 책 데이터는 개인 기록이므로 올리지 않습니다.

[컨테이너]

```
echo "books.json" >> .gitignore
python verify.py
git add -A
git status
git commit -m "2장: 책 등록, 목록, JSON 저장 (R1, R2, R5)"
```

`git status` 에 `books.json` 이 없어야 합니다. `.gitignore` 에 있으므로 보이지 않는 것이 정상입니다.

3. 확인하기

- ■ 빨강(실패)을 보고 원인을 말할 수 있다.
- ■ `python verify.py` 가 "검증 통과"로 끝났다.
- ■ `books.json` 을 열어 보고 한글이 깨지지 않았음을 확인했다.
- ■ 저장 커밋이 한 줄 생겼다.

4. 자주 만나는 오류

증상	해결
`ModuleNotFoundError: No module named 'library'`	`library.py` 가 없거나 폴더가 다르다. `ls` 로 확인한다.
`books.json` 의 한글이 `\uXXXX` 로 보인다	`ensure_ascii=False` 를 빠뜨렸다. AI 에게 `save_books` 를 고치게 한다.
저장한 파일을 읽을 때 `JSONDecodeError`	파일을 VS Code에서 직접 고쳐서 형식이 깨졌다. 파일을 지우고 다시 저장한다.

5. 확인 문제

1. 메모리에만 있던 데이터와 파일에 저장한 데이터의 차이를 한 문장으로 쓰시오.
2. JSON 의 `null` 은 파이썬의 무엇에 해당하나?
3. `books.json` 을 저장소에 올리지 않는 이유는 무엇인가?

2단계 3장. 대출과 반납: 여러 기능을 나눠 맡기기

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	2장 완료 (`library.py` 에 책 등록과 저장이 있다)
결과	대출과 반납을 AI 에게 맡기고, 상태 규칙이 지켜졌는지 확인해 저장한다

학습 목표

1. 큰 기능을 작은 지시로 나누는 이유를 말한다.
2. 상태 규칙(대출 중이면 빌릴 수 없다)이 코드에서 어떻게 보이는지 찾는다.
3. 테스트가 정한 기준을 AI 가 지키는지 확인한다.

1. 개념 — 한 번에 맡기지 않는다

"도서 대출 프로그램 전부 만들어 줘"라고 하면 AI 는 많은 것을 한꺼번에 만듭니다. 그러면 무엇이 잘못됐는지 찾기 어렵습니다.

작게 나누면

- 바뀐 코드가 적어서 내가 읽고 이해할 수 있습니다.
- 문제가 생기면 그 지시만 되돌리면 됩니다.
- 테스트가 어느 지시에서 깨졌는지 바로 알 수 있습니다.

> 2장에서 책 등록과 저장을 맡겼습니다. 이번 장은 대출과 반납만 맡깁니다.

2. 따라 하기

1단계. 상태 규칙 테스트 읽기 (20분)

[컨테이너]

```
cd /workspace/library
cp /tutorial/templates/stage2/tests/test_lending.py tests/
# (Windows) tests\test_lending.py 을(를) VS Code 로 열어 고칩니다
```

테스트 이름을 목록으로 적습니다. 그다음 아래 질문에 답합니다.

1. 대출 중인 책을 다시 빌리면 어떻게 되어야 하나? (테스트 이름을 찾는다)
2. 대출하지 않은 책을 반납하면 어떻게 되어야 하나?
3. 대출하면 책의 `borrower` 는 무엇이 되나?

2단계. 빨강 확인하기 (10분)

[컨테이너]

```
python -m pytest tests/test_lending.py
```

`lend_book`, `return_book` 이 없으므로 실패합니다. 실패 줄을 적어 둡니다.

3단계. AI 에게 맡기기 (40분)

[컨테이너]

```
cp /tutorial/templates/stage2/TASK_대출반납.md .
opencode
```

■ [AI] OpenCode 입력 칸

```
TASK_대출반납.md 를 읽고 그 지시대로 작업해 줘. 먼저 계획만 말하고 내 승인을 기다려.
```

계획에서 확인할 것:

■ 연체나 날짜 계산을 넣겠다는 말이 없는가?

■ `tests/` 를 고치겠다는 말이 없는가?

■ 두 함수가 모두 계획에 있는가? (`lend_book`, `return_book`)

맞으면 `진행` 이라고 답합니다.

4단계. 상태 규칙 직접 확인하기 (20분)

AI 가 끝나면 `/exit` 로 나옵니다. 테스트를 돌립니다.

[컨테이너]

```
python -m pytest tests/test_books.py tests/test_lending.py
```

모두 통과해야 합니다. 그다음 테스트가 말로 한 규칙을 **직접** 해 봅니다.

[컨테이너]

```
python -c "from library import *; b=[]; add_book(b,'데미안','헤세'); lend_book(b,1,'철수'); lend_book(b,1,'영희')"
```

오류가 나와야 합니다. 나오지 않으면 규칙이 지켜지지 않은 것입니다. 원인을 AI 에게 설명하게 합니다.

반납도 해 봅니다.

[컨테이너]

```
python -c "from library import *; b=[]; add_book(b,'데미안','헤세'); return_book(b,1)"
```

대출하지 않은 책을 반납하므로 오류가 나와야 합니다.

5단계. 규칙을 AI 에게 설명시키기 (15분)

파일을 고치지 말고 설명만 하게 합니다.

■ [AI] OpenCode 입력 칸

```
파일은 고치지 말고 설명만 해 줘. library.py 의 lend_book 에서 "이미 대출 중인 책" 을 확인하는 줄이 무슨 일을 해? 그 줄이 없으면 어떤 테
  스트가 실패해?
```

설명을 읽고, 테스트 이름과 맞는지 확인합니다.

6단계. 저장하기 (10분)

[컨테이너]

```
python verify.py
git add -A
git status
git commit -m "3장: 대출과 반납, 상태 규칙 (R3, R4)"
```

3. 확인하기

- ■ 두 테스트 파일이 모두 통과했다.
- ■ 이미 대출 중인 책을 다시 빌리면 오류가 나는 것을 직접 봤다.
- ■ 저장 커밋이 한 줄 생겼다.

4. 자주 만나는 오류

증상	해결
`AttributeError` 또는 `NameError`	AI 가 `_find` 함수를 빠뜨렸다. 실패 줄을 붙여 주고 고치게 한다.
책이 두 번 빌려진다	대출 중 확인 줄이 없다. 3단계의 확인 명령을 다시 실행해 증거를 보여 준다.
`tests/` 의 파일이 바뀌었다	`diff` 로 확인하고, AI 에게 원래대로 되돌리게 한다.

5. 확인 문제

1. 2장과 3장을 나눠 맡긴 이유를 한 문장으로 쓰시오.
2. 대출 중인 책을 반납하면 `status` 와 `borrower` 는 각각 무엇이 되나?
3. 테스트가 통과했는데도 내가 직접 규칙을 확인한 이유는 무엇인가?

2단계 4장. AI 가 만든 실수 찾기

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	3장까지 완료 (또는 `examples/stage2/library_bugged.py` 를 사용)
결과	AI 가 만든 초안의 실수를 테스트로 잡고, 테스트가 못 잡는 실수를 사람이 찾아 테스트를 추가한다

학습 목표

1. 테스트가 잡는 실수와 못 잡는 실수가 있다는 것을 안다.
2. 실패 출력을 읽어 어떤 약속이 깨졌는지 찾는다.
3. 테스트에 빠진 약속을 사람이 직접 추가한다.

1. 개념 — 테스트는 약속만 확인한다

테스트는 **적힌 약속만** 확인합니다. 약속에 없는 실수는 테스트가 통과해도 남아 있습니다.

종류	예	누가 찾나
약속이 있고, 지켜지지 않음	이미 대출 중인 책을 또 빌릴 수 있다	테스트가 실패로 알려 준다
약속이 없음	반납해도 빌린 사람 이름이 남는다	사람이 직접 보고 찾는다

> 이 장의 목표는 "테스트가 통과했으니 됐다"가 틀린 이유를 직접 보는 것입니다.

2. 따라 하기

1단계. 초안 가져오기 (10분)

이 장에서는 AI 가 만든 것처럼 준비된 초안을 씁니다. (3장까지 직접 만든 `library.py` 는 잠시 보관합니다.)

[컨테이너]

```
cd /workspace/library
cp library.py library_mine.py
cp /tutorial/examples/stage2/library_bugged.py library.py
ls -la
```

> `library_mine.py` 는 나중에 비교용으로 씁니다. 이 장이 끝나면 되돌립니다.

2단계. 검증하기 (10분)

[컨테이너]

```
python -m pytest tests/test_books.py tests/test_lending.py
```

몇 개가 실패합니다. 실패 목록을 적습니다.

```
FAILED ...
```

```
FAILED ...
(통과한 개수)
```

3단계. 실패 출력 읽기 (20분)

실패한 테스트 두 개의 이름과 assert 줄을 읽습니다.

실패한 테스트	깨진 약속 (테스트가 말하는 것)	원인으로 짐작되는 줄

> 테스트 이름이 바로 약속입니다. test_lend_lent_book_raises 는 "이미 대출 중인 책은 빌릴 수 없다"입니다.

4단계. AI 에게 고치게 하기 (25분)

[컨테이너]

```
opencode
```

■ [AI] OpenCode 입력 칸

```
python -m pytest 를 실행하면 두 테스트가 실패해. 실패한 테스트 이름과 아래 출력을 보고, 원인을 먼저 설명해 줘.
(실패 출력을 붙여 넣는다)
내가 "진행"이라고 하면 library.py 만 고쳐 줘. tests/ 는 건드리지 마.
```

계획을 검토하고 진행 이라고 답합니다. 끝나면 /exit.

[컨테이너]

```
python -m pytest tests/test_books.py tests/test_lending.py
```

이제 통과해야 합니다. 여기서 "끝났다"고 생각하면 안 됩니다.

5단계. 테스트가 못 잡은 실수 찾기 (25분)

테스트가 모두 통과한 상태에서, 대출과 반납을 한 번 해 봅니다.

[컨테이너]

```
python -c "from library import *; b=[]; add_book(b,'데미안','헤세'); lend_book(b,1,'철수'); print(return_book(b,1))"
```

결과를 읽습니다.

```
{'id': 1, ..., 'status': 'available', 'borrower': '철수'}
```

반납했는데 빌린 사람 이름(borrower)이 남아 있습니다. 1장의 요구사항 R4 에는 "borrower 를 None 으로 바꾼다"가 적혀 있습니다. 그런데 테스트에는 이 약속이 없습니다. 테스트는 status 만 확인하기 때문입니다.

이것이 **테스트가 못 잡은 실수**입니다.

6단계. 빠진 약속을 사람이 추가하기 (20분)

이번에는 **사람이** 테스트를 씁니다. AI 에게 맡기지 않습니다.

[Windows]

```
# (Windows) tests\test_lending.py 을(를) VS Code 로 열어 고칩니다
```

파일 맨 아래에 다음을 추가하고 저장합니다.

```


def test_return_clears_borrower():
```

```
    # 반납하면 빌린 사람 이름을 지운다. (요구사항 R4)
    books = make_books()
    lend_book(books, 1, "철수")
    book = return_book(books, 1)
    assert book["borrower"] is None
```

테스트를 실행합니다.

[컨테이너]

```
python -m pytest tests/test_lending.py::test_return_clears_borrower
```

실패해야 합니다(빨강). 이것이 새로 찾은 실수를 잡는 약속입니다.

7단계. AI 에게 고치게 하기 (20분)

■ **[AI] OpenCode 입력 칸**

```
tests/test_lending.py 의 test_return_clears_borrower 가 실패해. 반납할 때 borrower 를 None 으로 바꾸는 줄을 library.py 의 return_book 에
  추가해 줘. 계획을 먼저 말하고 승인을 기다려. tests/ 는 고치지 마.
```

진행 이라고 답하고, 끝나면 확인합니다.

[컨테이너]

```
python verify.py
```

"검증 통과"가 나오면 실수가 고쳐진 것입니다.

8단계. 비교하고 저장하기 (15분)

3장에서 직접 만든 `library_mine.py` 와 지금 고친 `library.py` 의 `return_book` 을 비교합니다.

[Windows]

```
# (Windows) library_mine.py 을(를) VS Code 로 열어 고칩니다
# (Windows) library.py 을(를) VS Code 로 열어 고칩니다
```

두 파일의 `return_book` 에서 차이를 한 가지 적습니다. 차이를 적은 뒤 비교용 파일은 지웁니다.

[컨테이너]

```
rm -f library_mine.py
python verify.py
git add -A
git status
git commit -m "4장: 테스트가 못 잡은 반납 실수를 테스트로 추가 (R4)"
```

> `library_mine.py` 는 저장소에 올리지 않습니다. 지운 뒤 커밋하는 이유입니다.

3. 확인하기

- ■ 실패 두 개를 보고, 깨진 약속의 이름을 말할 수 있다.
- ■ 테스트가 통과한 뒤에도 실수가 남아 있는 것을 직접 봤다.
- ■ 빠진 약속을 사람이 테스트로 추가했다.
- ■ `tests/` 안의 기존 테스트는 바뀌지 않았다.

4. 자주 만나는 오류

증상	해결
실패가 두 개가 아니다	초안을 바꿨는지 확인한다. `cp` 를 다시 실행한다.
AI 가 테스트를 고쳐서 통과시켰다	권한이 막아야 한다. 막히지 않았다면 `diff` 로 확인하고 원래대로 되돌린다.
새 테스트를 추가했는데 실패하지 않는다	그 테스트가 확인하는 줄을 다시 본다. `make_books()` 와 `lend_book` 호출 순서를 확인한다.

5. 확인 문제

1. 테스트가 통과했는데도 실수가 남는 이유를 "약속"이라는 낱말로 설명하시오.
2. `test_lend_lent_book_raises` 라는 이름이 약속을 어떻게 말하고 있는가?
3. 이번 장에서 빠진 약속을 AI 가 아니라 사람이 쓴 이유는 무엇인가?
4. 요구사항 R4 와 테스트 중 어느 쪽이 더 완전했나? 어떻게 맞춰야 하나?

2단계 5장. 요구사항이 바뀌었을 때

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	4장까지 완료. `templates/stage2/CHANGE_요청서.md`, `templates/stage2/tests/test_lending_v2.py`
결과	요구사항 변경을 변경 요청서로 정리하고, 기준을 사람이 바꾼 뒤 AI 에게 구현시킨다

학습 목표

1. 요구사항이 바뀌면 테스트가 어떻게 깨지는지 직접 본다.
2. 변경 요청서를 쓰고, 승인하는 사람과 바꾸는 사람을 나눈다.
3. 기준(테스트)을 바꾸는 일과 구현을 바꾸는 일을 구분한다.

1. 개념 — 바뀔 때는 기준부터

프로젝트 중간에 요구사항이 바뀌는 것은 흔합니다. 예: "대출 기간 14일을 기록해 주세요." 이때 무턱대고 코드부터 고치면 두 가지 문제가 생깁니다.

- 기준이 바뀌었는데 테스트가 옛 기준을 붙들고 있어서 실패합니다. 실패가 "잘못된 구현" 때문인지 "옛 기준" 때문인지 알 수 없습니다.
- AI 가 기준(테스트)까지 바꿔서 통과시키면, 검증이 아무 의미도 없어집니다.

그래서 순서가 있습니다.

```
① 변경 요청서를 쓴다 (무엇이, 왜 바뀌나)
② 사람이 승인한다
③ 사람이 기준(테스트)을 바꾼다
④ AI 가 구현을 바꾼다 (기준은 고치지 않는다)
⑤ 검증하고 저장한다
```

2. 따라 하기

1단계. 변경 전 상태 확인하기 (10분)

[컨테이너]

```
cd /workspace/library
python verify.py
git status
```

"검증 통과"여야 합니다. 이 상태가 **변경 전의 기준점**입니다.

2단계. 변경 요청서 쓰기 (20분)

[컨테이너]

```
cp /tutorial/templates/stage2/CHANGE_요청서.md .
# (Windows) CHANGE_요청서.md 을(를) VS Code 로 열어 고칩니다
```

요청서의 1번 칸을 채웁니다.

- 요구사항 번호: R3, R4
- 무엇이 바뀌나: 대출하면 `loan_days` 칸에 `14` 가 들어가고, 반납하면 지운다.
- 왜 바꾸나: 대출 기간을 알아야 한다. (이유를 한 문장으로 쓴다)

요청서의 2번 칸(영향을 받는 테스트)은 **직접 찾아 적습니다.** 힌트: 책 모양이 달라지는 테스트를 찾습니다.

3단계. 영향 확인하기 (10분)

변경을 구현하기 전에, 새 기준을 옛 구현에 적용해 봅니다. 이것이 "무엇이 깨지는가"를 미리 보는 방법입니다.

[컨테이너]

```
cp /tutorial/templates/stage2/tests/test_lending_v2.py tests/test_lending_v2.py
python -m pytest tests/test_lending_v2.py
```

실패가 나와야 합니다. 몇 개가 실패하는지, 어떤 이름인지 적습니다.

실패한 테스트	실패 이유 (assert 줄을 읽고 한 문장으로)

> 실패를 보고 "구현이 틀렸다"고 결론 내리면 안 됩니다. 기준이 바뀌었으니 실패는 정상입니다.

4단계. 승인하고 기준 바꾸기 (15분)

요청서의 4번 칸(승인)을 채웁니다. 승인은 요청한 사람과 기준을 바꾸는 사람이 함께 확인합니다. 이 과정에서는 **내가 두 역할을 모두 맡지만, 순서는 지킵니다.**

- ■ 변경 이유를 다시 읽었다.
- ■ 영향 받는 테스트를 확인했다.
- ■ 승인 이름과 날짜를 적었다.

승인이 되었으면 기준을 바꿉니다. 옛 테스트 파일을 새 기준으로 바꿉니다.

[컨테이너]

```
cp -f tests/test_lending_v2.py tests/test_lending.py
rm -f tests/test_lending_v2.py
git diff --stat
```

`tests/test_lending.py` 가 바뀐 것이 보이면 정상입니다. 이 변경은 **사람이 한 것**이므로, 커밋 메시지에 "기준 변경(승인됨)"이라고 적습니다.

5단계. AI 에게 구현하게 하기 (30분)

[컨테이너]

```
opencode
```

■ [AI] OpenCode 입력 칸

```
CHANGE_요청서.md 의 1번 변경을 구현해 줘. tests/ 는 이미 새 기준으로 바뀌어 있으니 고치지 마.
계획을 먼저 말하고 내가 "진행"이라고 하면 library.py 만 고쳐 줘.
```

계획을 검토합니다.

- ■ `library.py` 만 고치겠다고 했는가?
- ■ `tests/` 를 고치겠다는 말이 없는가?
- ■ `loan_days` 를 대출할 때 넣고, 반납할 때 지우겠다고 했는가?

맞으면 `진행`. 끝나면 `/exit`.

6단계. 검증하고 저장하기 (15분)

[컨테이너]

```
python verify.py
git status
```

■ "검증 통과"가 나온다.

■ `git status` 에 `tests/` 가 바뀐 것으로 보이는 것은 **사람이 바꾼 기준** 하나뿐이다.

저장은 변경 이유를 커밋 메시지에 적습니다.

[컨테이너]

```
git add -A
git commit -m "5장: 대출 기간 14일 기록 (요구사항 변경 승인됨, 기준은 사람이 변경, R3·R4)"
```

3. 확인하기

■ 새 기준을 옛 구현에 적용했을 때 실패하는 것을 직접 봤다.

■ 기준 변경을 사람이 했고, AI 는 구현만 고쳤다.

■ 커밋 메시지에 변경 이유와 승인 여부가 들어 있다.

4. 자주 만나는 오류

증상	해결
AI 가 `test_lending.py` 를 고쳐서 통과시켰다	기준 변경은 사람이 한다. `diff` 로 확인하고 AI 에게 원래대로 되돌리게 한다.
구현 후에도 한두 개 실패한다	실패 줄을 읽고, 요구사항 R3 또는 R4 의 어느 문장인지 찾는다.
`cp` 가 파일을 덮어쓰지 않는다	`cp -f` 로 다시 복사한다.

5. 확인 문제

1. 기준을 먼저 바꾸고 구현을 나중에 바꾸는 이유는 무엇인가?
2. 새 기준을 옛 구현에 적용했을 때 생긴 실패를 "구현의 잘못"이라고 하면 왜 틀린가?
3. 요청서의 "승인" 칸이 없으면 어떤 위험이 생기는가?

2단계 6장. 잘못된 변경을 되돌리기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	5장까지 완료 (커밋 기록이 여러 줄 있다)
결과	저장 전 변경과 저장 후 변경을 각각 안전하게 되돌리고, 되돌린 뒤 검증한다

학습 목표

1. "검증 통과"와 "요구사항 범위 안"은 다르다는 것을 직접 본다.
2. 저장하기 전의 잘못된 변경을 `git restore` 로 되돌린다.
3. 이미 저장한 잘못된 변경을 `git revert` 로 되돌린다. (기록을 지우지 않는다)

1. 개념 — 되돌리는 방법은 두 가지다

상황	방법	무슨 일이 생기나
아직 저장하지 않았다	`git restore`	바뀐 내용이 마지막 저장 상태로 돌아간다
이미 저장했다	`git revert`	되돌리는 새 저장을 하나 더 만든다. 원래 기록은 남는다

하지 않을 것: `git reset --hard` 는 기록과 내용을 함께 지웁니다. 실수로 쓰면 되찾기 어렵습니다. 이 과정의 OpenCode 권한은 `git reset` 을 막고 있습니다.

되돌리기도 검증해야 합니다. 되돌린 뒤 `python verify.py` 가 통과하는지 확인합니다.

2. 따라 하기

1단계. 범위를 벗어난 변경 만들기 (20분)

일부러 요구사항에 없는 기능을 AI 에게 시킵니다. 계획 검토를 건너뛰라고 말하는 것이 핵심입니다. 이것은 **실수를 재현하는 연습**입니다.

[컨테이너]

```
cd /workspace/library
git status
opencode
```

■ [AI] OpenCode 입력 칸

```
library.py 에 책 삭제 기능 delete_book(books, book_id) 를 추가해 줘. 계획 없이 바로 진행해 줘.
```

AI 가 끝나면 `/exit`.

[컨테이너]

```
python verify.py
git status
git diff
```

- `python verify.py` 가 "검증 통과"로 나온다. (기존 테스트는 깨지지 않았다)
- 그런데 `delete_book` 은 **요구사항 어디에도 없다.** ("만들지 않는 것"에도 없다.)

> **검증 통과는 "약속을 지켰다"이지, "약속 밖의 일을 하지 않았다"가 아닙니다.** 이것이 이 장의 첫 번째 교훈입니다. 범위는 지시서와 요구사항으로 따로 확인합니다.

2단계. 저장 전 변경 되돌리기 (20분)

`git status` 에서 바뀐 파일을 확인합니다. `library.py` 가 수정됐다면 아래로 되돌립니다.

[컨테이너]

```
git restore library.py
python verify.py
git status
```

- `library.py` 가 마지막 저장 상태로 돌아갔다.
- `delete_book` 이 사라졌는지 확인한다. (`grep delete_book library.py` 가 아무것도 찾지 못해야 한다)
- "검증 통과"가 나온다.

> 새 파일이 생겼다면 `git restore` 로는 지워지지 않습니다. `git status` 의 `Untracked files` 에 나온 파일을 직접 지웁니다.

3단계. 저장한 뒤 실수 되돌리기 (25분)

이번에는 실수를 **저장한 뒤**에 되돌립니다. 먼저 같은 실수를 다시 만들고 저장합니다.

[컨테이너]

```
opencode
```

■ [AI] OpenCode 입력 칸

```
library.py 에 책 삭제 기능 delete_book(books, book_id) 를 추가해 줘. 계획 없이 바로 진행해 줘.
```

`/exit` 로 나온 뒤 저장합니다.

[컨테이너]

```
python verify.py
git add -A
git commit -m "실험: 범위 밖 기능 delete_book 저장"
git log --oneline
```

이제 실수가 기록에 남았습니다. 되돌립니다.

[컨테이너]

```
git revert --no-edit HEAD
git log --oneline
```

`git log` 를 보면 "실험"이라는 커밋과 "Revert" 커밋이 **둘 다** 남아 있습니다. 이것이 `revert` 입니다. 기록은 지워지지 않고, 되돌린 사실이 새 기록으로 쌓입니다.

되돌린 뒤 검증합니다.

[컨테이너]

```
python verify.py
grep delete_book library.py
```

- "검증 통과"가 나온다.
- `delete_book` 이 `library.py` 에 없다.

4단계. 되돌리기와 되돌리지 말기 (15분)

다음 상황에서 무엇을 하겠는지 적습니다.

상황	무엇을 하나 (`restore` / `revert` / 아무것도 안 함)	이유
방금 AI 가 바꾼 파일을 저장하기 전에 아니라고 판단했다		
이미 저장된 커밋에 버그가 있다는 것을 어제 발견했다		
테스트가 실패하지만 원인을 아직 모른다		
3번 시도해도 검증이 통과하지 않는다		

> 힌트: 저장 여부와 "기록을 남겨야 하는가"를 먼저 생각합니다. 원인을 모르는 상태에서 되돌리는 것이 나은지, 원인을 먼저 찾는 것이 나은지도 함께 생각합니다. 답은 강사와 함께 확인합니다.

5단계. 기록 정리 확인하기 (10분)

[컨테이너]

```
git log --oneline
git status
```

- `git status` 가 깨끗하다. (`nothing to commit` 또는 비슷한 문장)
- 실험 커밋과 되돌리기 커밋이 모두 기록에 있다. (지우지 않았다)

3. 확인하기

- 범위 밖 변경을 검증이 통과시켰다는 것을 직접 봤다.
- 저장 전 변경을 `git restore` 로 되돌렸다.
- 저장 후 변경을 `git revert` 로 되돌렸고, 기록이 남아 있는 것을 봤다.
- 4단계 표를 채웠다.

4. 자주 만나는 오류

증상	해결
`git revert` 가 편집기를 연다	`--no-edit` 를 빠뜨렸다. 편집기를 닫으려면 `Esc` 를 누르고 `:q!` 를 친다. (VS Code 면 저장하고 닫는다)
`git revert` 가 충돌을 알린다	되돌릴 커밋 이후에 같은 줄을 또 고쳤다. `git revert --abort` 로 취소하고 `git log` 로 순서를 확인한다.
`git restore` 후에도 파일이 남아 있다	새로 만들어진 파일이다. `git status` 에서 찾아 직접 지운다.
되돌렸는데 검증이 실패한다	되돌리기 전 다른 변경이 섞여 있었을 수 있다. `git log` 와 `git diff` 로 확인한다.

5. 확인 문제

1. "검증 통과"와 "범위 안에 있다"의 차이를 한 문장으로 쓰시오.
2. `git restore` 와 `git revert` 의 차이를 "기록"이라는 낱말로 설명하시오.
3. `git reset --hard` 를 쓰면 안 되는 이유는 무엇인가?
4. 되돌린 뒤에도 `python verify.py` 를 다시 해야 하는 이유는 무엇인가?

2단계 7장. 미니 프로젝트: 도서 대출 목록 완성

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간 30분
준비물	1~6장 완료
결과	도서 대출 목록을 처음부터 다시 만들고, 검증과 저장을 거쳐 완성한다. 과정 기록을 쓴다.

학습 목표

1. 1~6장의 요소(요구사항, 지시서, 기준, 변경, 검증, 되돌리기)를 한 번에 쓴다.
2. 요구사항 변경이 반영된 최종 기준을 사람이 확인하고 AI 에게 구현시킨다.
3. 범위와 검증, 두 가지를 모두 확인하고 저장한다.

1. 개념 — 지금까지의 고리

```
① 요구사항 (REQUIREMENTS.md, 변경 반영 완료)   1장 · 5장
② 기준 (tests/, 사람이 준비)                  2장 · 3장 · 5장
③ 작업 지시서 (TASK_미니프로젝트.md)           3장
④ 규칙과 권한 (AGENTS.md, opencode.jsonc)     1단계 4장
⑤ 검증과 범위 확인 (verify.py, 요구사항 대조)  4장 · 6장
⑥ 저장 또는 되돌리기 (git commit, revert)     2장 · 6장
```

이번 장은 **새 폴더에서 처음부터** 합니다. 2~6장의 `library.py` 를 가져오지 않습니다.

2. 따라 하기

1단계. 새 폴더와 최종 기준 준비하기 (15분)

[컨테이너]

```
mkdir -p /workspace/library-final
mkdir -p /workspace/library-final/tests
cd /workspace/library-final
git init
git config core.hooksPath .githooks

T="/tutorial/templates"
cp "$T/stage2/tests/test_books.py" tests/
cp "$T/stage2/tests/test_lending_v2.py" tests/test_lending.py
cp "$T/stage2/REQUIREMENTS.md" .
cp "$T/stage2/TASK_미니프로젝트.md" .
cp "$T/stage2/pytest.ini" .
cp "$T/verify.py" .
cp "$T/opencode.jsonc" .
cp "$T/AGENTS_빈칸.md" AGENTS.md
mkdir -p .githooks
cp "$T/pre-commit" .githooks/pre-commit
cp "$T/.gitattributes" .
printf 'books.json\n.env\n' > .gitignore
ls -la tests
```

> `tests\test_lending.py` 는 2차(대출 기간 포함) 기준입니다. 5장에서 승인된 최종 기준이기 때문입니다.

2단계. 요구사항 확인하기 (15분)

`REQUIREMENTS.md` 를 읽고 아래를 표시합니다.

- R3 의 대출에 `loan_days` 14 가 들어 있다.
- R4 의 반납에 `borrower` 를 `None` 으로 바꾸고, `loan_days` 를 지운다.
- "만들지 않는 것"에 책 삭제와 연체가 있다.

이 요구사항과 `tests\test_lending.py` 가 같은 말을 하는지 대조합니다. 다르면 **요구사항이 아니라 기준을 바꿀지 먼저 결정합니다.** (변경 요청서가 필요합니다)

3단계. AI 에게 맡기기 (30분)

[컨테이너]

```
opencode
```

[AI] OpenCode 입력 칸

```
TASK_미니프로젝트.md 를 읽고 그 지시대로 작업해 줘. REQUIREMENTS.md 와 tests/ 를 기준으로 삼는다. 먼저 계획만 말하고 내 승인을 기다려.
```

계획을 검토합니다.

- `library.py` 하나만 만들겠다고 했는가?
- R1~R5 가 모두 계획에 있는가?
- 책 삭제, 연체, 웹 화면, 데이터베이스가 있는가? (있으면 뺀다)

맞으면 진행. 끝나면 `/exit`.

4단계. 검증 두 가지 하기 (25분)

(1) 기계 검증

[컨테이너]

```
python verify.py
```

"검증 통과"가 나와야 합니다.

(2) 범위 검증 (사람이 한다)

`library.py` 에 함수 이름을 모두 나열합니다.

[컨테이너]

```
grep "^def " library.py
```

요구사항의 기능과 하나씩 대조합니다.

함수	요구사항	있어야 하나?
add_book	R1	
list_books	R2	
lend_book	R3	
return_book	R4	
save_books, load_books	R5	
(그 밖의 함수)	없음	있으면 되돌린다

(3) 직접 실행 확인

[컨테이너]

```
python -c "from library import *; b=[]; add_book(b,'데미안','헤세'); add_book(b,'노인과 바다','헤밍웨이'); lend_book(b,1,'철수'); print(
  list_books(b,'lent')); print(return_book(b,1)); print(list_books(b,'available'))"
```

■ 대출 중 목록에 "데미안"이 보인다.

■ 반납한 책의 `borrower` 가 `None` 이고, `loan_days` 가 없다.

■ 이미 대출 중인 책을 다시 빌리면 오류가 난다.

5단계. 문제가 있으면 되돌리거나 고치기 (15분)

범위 밖 함수가 있거나 직접 실행이 틀리면, 6장에서 배운 대로 합니다.

- 저장하지 않았다면: `git restore library.py` 후 지시서를 더 작게 나눠 다시 맡긴다.
- 저장했다면: `git revert` 로 되돌린다. (아직 저장하지 않았으므로 이 장에서는 해당 없음)

고칠 것이 있으면 AI 에게 말합니다.

■ [AI] OpenCode 입력 칸

```
범위 밖 함수 <이름> 이 있어. 요구사항에 없으니 지워 줘. 계획을 먼저 말하고, 내가 "진행"이라고 하면 library.py 만 고쳐 줘.
```

6단계. 저장하기 (10분)

[컨테이너]

```
python verify.py
git add -A
git status
git commit -m "7장: 도서 대출 목록 완성 (R1~R5, 검증과 범위 확인 완료)"
git log --oneline
```

`git status` 에 `books.json` 이 없어야 합니다. (`.gitignore` 에 있습니다)

7단계. 과정 기록 쓰기 (10분)

`기록.md` 를 만들고 아래를 채웁니다.

```
# 7장 기록

## 1. 이 프로젝트에서 바뀐 요구사항 한 가지와, 왜 기준을 사람이 바꿨는가

## 2. 검증 통과와 범위 확인이 다르다는 것을 보여 준 장면 한 가지

## 3. 되돌리기를 했다면 어떤 방법을 썼고, 왜 그 방법을 골랐는가 (없으면 "없음")

## 4. 내가 이해한 코드 세 문장
1.
2.
3.

## 5. 2단계를 마치고 1단계와 달라진 점 한 가지
```

3. 확인하기

■ 새 폴더에서 처음부터 만든 `library.py` 가 R1~R5 를 모두 지킨다.

■ `python verify.py` 가 "검증 통과"다.

■ 범위 밖 함수가 없다. (사람이 확인했다)

■ 저장 커밋이 한 줄 생겼다.

■ `기록.md` 를 썼다.

4. 막혔을 때

증상	해결
테스트가 `loan_days` 때문에 실패한다	요구사항 R3 와 `tests\test_lending.py` 가 같은 2차 기준인지 확인한다.
AI 가 `tests/` 를 고치려 한다	권한 설정이 막는다. 막히지 않았다면 `diff` 로 확인하고 되돌린다.
계획에 연체 계산이 들어 있다	TASK 의 "하지 말 것"을 가리키며 빼게 한다.

> 이 장에서도 **정답 코드는 제공하지 않습니다.** 요구사항과 테스트를 기준으로 AI 와 대화하며 맞춰 갑니다.

5. 확인 문제

1. 최종 기준에 `loan_days` 가 들어간 것은 어떤 절차를 거쳐서였나?
2. 이 프로젝트에서 "검증 통과"만으로 끝내면 안 되는 이유를 예를 들어 설명하시오.
3. 2단계에서 가장 중요했던 하네스 요소는 무엇이었다고 생각하는가? 이유와 함께 쓰시오.

다음 단계

이것으로 2단계가 끝납니다. 3단계는 같은 흐름을 **FastAPI 웹 API, PostgreSQL 데이터베이스, Docker** 로 옮겨 가는 3일 과정입니다. 저장소 harness-crud-lab 에서 이어갑니다.

부록. 템플릿과 예제

수업에서 복사해 쓰는 파일을 그대로 실었다. 주석을 따라 읽는다.

A-1. 도서 대출 요구사항 최종본 (7장)

```
# 요구사항: 도서 대출 목록 (2단계 7장, 최종 버전)

## 한 문장 요약
책을 등록하고, 대출하고, 반납하고, 목록을 보는 명령줄 프로그램. 책 목록은 JSON 파일에 저장되어 프로그램을 껐다 켜도 남는다.

## 책 한 권의 모양
| 항목 | 설명 |
|---|---|
| `id` | 1, 2, 3 … 순서대로 붙는 번호. 지워지는 기능이 없으므로 항상 새 번호를 쓴다. |
| `title` | 책 제목. 공백만 있으면 안 된다. |
| `author` | 지은이. 비워 둘 수 있다. |
| `status` | `"available"`(대출 가능) 또는 `"lent"`(대출 중). 처음에는 `"available"`. |
| `borrower` | 빌려 간 사람 이름. 대출 중이면 이름이, 대출 가능하면 `None`. |
| `loan_days` | 대출 기간(일). 대출 중일 때만 `14` 가 들어 있다. 반납하면 이 칸을 지운다. |

## 기능 요구사항

### R1. 책 등록
- `add_book(books, title, author)` 을 부르면 새 책을 목록 끝에 넣고, 그 책을 돌려준다.
- 제목이 공백뿐이면 `ValueError` 를 일으킨다.

### R2. 목록 보기
- `list_books(books)` 은 모든 책을 돌려준다.
- `list_books(books, "available")` 또는 `list_books(books, "lent")` 은 그 상태의 책만 돌려준다.

### R3. 대출
- `lend_book(books, book_id, borrower)` 을 부르면 그 책의 `status` 를 `"lent"`, `borrower` 를 빌리는 사람 이름으로 바꾸고, `loan_days` 를 `14` 로 넣는다.
- 없는 번호면 `ValueError`.
- 이미 대출 중인 책이면 `ValueError`.
- 빌리는 사람 이름이 공백뿐이면 `ValueError`.

### R4. 반납
- `return_book(books, book_id)` 을 부르면 그 책의 `status` 를 `"available"` 로 바꾸고, `borrower` 를 `None` 으로, `loan_days` 칸을 지운다.
- 없는 번호면 `ValueError`.
- 대출 중이 아닌 책이면 `ValueError`.

### R5. 파일 저장과 읽기
- `save_books(books, path)` 은 책 목록을 JSON 파일로 저장한다. 한글이 깨지지 않는다.
- `load_books(path)` 은 JSON 파일을 읽어 목록을 돌려준다.
- 파일이 없으면 빈 목록 `[]` 을 돌려준다.

### R6. 명령줄에서 쓰기 (선택)
- `python library.py` 뒤에 `add`, `list`, `lend`, `return` 을 붙여 쓸 수 있다.
- 목록은 번호, 상태, 제목, 지은이, 빌린 사람을 한 줄씩 보여 준다.

## 만들지 않는 것
- 연체 계산, 날짜 계산 (대출 기간은 숫자 14 로만 기록한다)
- 회원 관리, 여러 사람이 동시에 쓰는 기능
- 책 삭제, 책 정보 수정
- 웹 화면, 데이터베이스
```

A-2. 요구사항 변경 요청서 (5장)

```
# 요구사항 변경 요청서 (2단계 5장)

> 요구사항이 바뀌면, 코드를 고치기 전에 이 요청서를 먼저 씁니다.
> 바뀌는 것은 사람이 정하고, 승인도 사람이 합니다. AI 는 승인된 변경만 구현합니다.

## 1. 바뀌는 요구사항
- 요구사항 번호: R3 (대출), R4 (반납)
- 무엇이 바뀌나: 대출하면 `loan_days` 칸에 `14` 가 함께 기록된다. 반납하면 그 칸을 지운다.
- 왜 바꾸나: 도서관에서 대출 기간을 알아야 한다. (이 요청의 이유를 한 문장으로 적는다)

## 2. 영향을 받는 테스트
- `tests/test_lending.py` 의 `test_lend_book_marks_lent` : 기대하는 책 모양이 달라진다.
- 그 밖의 테스트는 이 변경으로 바뀌지 않아야 한다.

## 3. 테스트 기준을 어떻게 바꾸나
- 새 기준: `templates/stage2/tests/test_lending_v2.py` 를 `tests/test_lending.py` 자리에 넣는다.
- 기준을 바꾸는 것은 **사람이 한다.** AI 에게 맡기지 않는다.

## 4. 승인
- [ ] 요청한 사람이 이 변경의 이유를 읽었다.
- [ ] 기준(테스트)을 바꾸는 일을 사람이 했다.
- [ ] 승인 날짜와 이름: _____________________

---

## 구현 지시 (승인 뒤에만 쓴다)
```

CHANGE_요청서.md 의 1번 변경을 구현해 줘. tests/ 는 이미 새 기준으로 바뀌어 있으니 고치지 마. 계획을 먼저 말하고 내가 "진행"이라고 하면 library.py 만 고쳐 줘.

A-3. 책 저장 작업 지시서 (2장)

```
# 작업 지시서: 책 등록과 파일 저장 (2단계 2장)

## 1. 목표
책을 등록하고, 목록을 보고, JSON 파일에 저장하고 읽어 올 수 있는 `library.py` 의 앞부분을 만든다.

## 2. 요구사항
`REQUIREMENTS.md` 의 **R1, R2, R5** 를 구현한다. (R3, R4 는 이번에 만들지 않는다)

## 3. 범위
- 새로 만드는 파일: `library.py` 하나
- 고쳐도 되는 파일: `library.py` 하나뿐이다.
- 고치면 안 되는 파일: `tests/` 폴더 전체, `verify.py`, `AGENTS.md`, `opencode.jsonc`

## 4. 하지 말 것
- `lend_book`, `return_book` 을 만들지 않는다. (3장에서 한다)
- 웹 화면, 데이터베이스, 외부 패키지를 쓰지 않는다. 파이썬 기본 기능만 쓴다.
- 책 삭제나 수정 기능을 덧붙이지 않는다.

## 5. 완료 조건
- [ ] `python -m pytest tests/test_books.py` 가 모두 통과한다.
- [ ] `library.py` 의 모든 함수에 한국어 주석이 있다.
- [ ] `tests/` 폴더는 바뀌지 않았다. (`Compare-Object` 로 원본과 비교했을 때 차이가 없다)

## 6. 진행 방법
`AGENTS.md` 의 작업 순서를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 구현한다.
끝나면 바꾼 파일과 통과한 테스트 개수를 보고한다.
```

A-4. 대출과 반납 작업 지시서 (3장)

```
# 작업 지시서: 대출과 반납 (2단계 3장)

## 1. 목표
책을 빌려 주고 돌려받는 기능을 `library.py` 에 추가한다. 상태 규칙을 지키는 것이 핵심이다.

## 2. 요구사항
`REQUIREMENTS.md` 의 **R3, R4** 를 구현한다. (R1, R2, R5 는 이미 되어 있다)

## 3. 범위
- 고쳐도 되는 파일: `library.py` 하나뿐이다.
- 고치면 안 되는 파일: `tests/` 폴더 전체, `verify.py`, `AGENTS.md`, `opencode.jsonc`

## 4. 하지 말 것
- 연체, 날짜 계산을 만들지 않는다. (대출 기간은 이번 장에서 넣지 않는다)
- 대출 기록을 따로 파일로 남기지 않는다.
- 테스트 기대값을 바꾸거나, 테스트를 건너뛰게 만들지 않는다.

## 5. 완료 조건
- [ ] `python -m pytest tests/test_books.py tests/test_lending.py` 가 모두 통과한다.
- [ ] `tests/` 폴더는 바뀌지 않았다.
- [ ] 이미 대출 중인 책을 다시 빌리려 하면 오류가 나고, 상태는 바뀌지 않는다. (내가 직접 확인한다)

## 6. 진행 방법
`AGENTS.md` 의 작업 순서를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 구현한다.
계획에 테스트 파일 수정이 들어 있으면 그 줄을 빼게 한 뒤 승인한다.
```

A-5. 미니 프로젝트 작업 지시서 (7장)

```
# 작업 지시서: 도서 대출 목록 완성 (2단계 7장)

## 1. 목표
`REQUIREMENTS.md` 의 R1~R5 를 모두 만족하는 `library.py` 를 완성한다. (R6 명령줄 기능은 선택)

## 2. 요구사항
`REQUIREMENTS.md` 의 **R1, R2, R3, R4, R5**. 대출 기간 `loan_days` 는 `CHANGE_요청서.md` 로 승인된 변경이다.

## 3. 범위
- 고쳐도 되는 파일: `library.py` 하나뿐이다.
- 고치면 안 되는 파일: `tests/` 폴더 전체, `verify.py`, `AGENTS.md`, `opencode.jsonc`, `REQUIREMENTS.md`

## 4. 하지 말 것
- 연체 계산, 날짜 계산, 책 삭제, 책 수정을 만들지 않는다.
- 웹 화면, 데이터베이스, 외부 패키지를 쓰지 않는다.
- 테스트 파일의 기대값을 바꾸지 않는다.

## 5. 완료 조건
- [ ] `python verify.py` 가 "검증 통과"로 끝난다.
- [ ] `tests/` 폴더는 템플릿 원본과 같다. (`Compare-Object` 로 확인)
- [ ] 대출 중인 책을 반납하면 `borrower` 가 `None` 이 되는지 내가 직접 확인했다.
- [ ] 모든 함수에 한국어 주석이 있다.

## 6. 진행 방법
`AGENTS.md` 의 작업 순서를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 구현한다.
끝나면 `AGENTS.md` 의 보고 형식으로 보고한다.
```

A-6. 책 등록과 저장 테스트 (2장)

```
"""
test_books.py - 책 등록, 목록, 저장 약속 (2단계 2장)

이 파일이 보여 주는 개념
  1) 기준(테스트)은 사람이 정한다. AI 는 이 파일을 고칠 수 없다.
  2) 저장 테스트는 임시 폴더(tmp_path)에만 파일을 쓴다. 내 진짜 파일을 건드리지 않는다.
"""

import pytest

from library import add_book, list_books, load_books, save_books


def test_add_book_creates_available_book():
    books = []
    book = add_book(books, "데미안", "헤세")
    assert book == {"id": 1, "title": "데미안", "author": "헤세", "status": "available", "borrower": None}
    assert books == [book]


def test_add_book_numbers_go_up():
    books = []
    add_book(books, "첫째", "가")
    second = add_book(books, "둘째", "나")
    assert second["id"] == 2


def test_add_book_rejects_blank_title():
    # 공백만 있는 제목은 받지 않는다.
    with pytest.raises(ValueError):
        add_book([], "   ", "헤세")


def test_list_books_filters_by_status():
    books = []
    add_book(books, "데미안", "헤세")
    assert list_books(books) == books
    assert list_books(books, "available") == books
    assert list_books(books, "lent") == []


def test_save_and_load_roundtrip(tmp_path):
    path = tmp_path / "books.json"  # 임시 폴더 안의 파일만 쓴다
    books = []
    add_book(books, "데미안", "헤세")
    save_books(books, path)
    assert load_books(path) == books


def test_load_missing_file_returns_empty_list(tmp_path):
    # 파일이 아직 없으면 빈 목록을 돌려준다. 오류를 내지 않는다.
    assert load_books(tmp_path / "없는파일.json") == []
```

A-7. 대출과 반납 테스트 1차 (3장)

```
"""
test_lending.py - 대출과 반납 약속 (2단계 3장, 1차 버전)

이 파일이 보여 주는 개념
  1) 상태 규칙: 대출 중인 책은 다시 대출할 수 없고, 대출하지 않은 책은 반납할 수 없다.
  2) 정상 경우와 잘못된 경우를 모두 약속한다.
  3) 4교시에 "반납하면 빌린 사람 이름을 지운다"는 약속이 빠져 있다는 것을 발견하게 된다.
"""

import pytest

from library import add_book, lend_book, return_book


def make_books():
    """테스트마다 책이 한 권 있는 목록을 만든다."""
    books = []
    add_book(books, "데미안", "헤세")
    return books


def test_lend_book_marks_lent():
    books = make_books()
    book = lend_book(books, 1, "철수")
    assert book == {"id": 1, "title": "데미안", "author": "헤세", "status": "lent", "borrower": "철수"}


def test_lend_lent_book_raises():
    # 이미 대출 중인 책은 다시 빌릴 수 없다.
    books = make_books()
    lend_book(books, 1, "철수")
    with pytest.raises(ValueError):
        lend_book(books, 1, "영희")


def test_lend_missing_book_raises():
    with pytest.raises(ValueError):
        lend_book(make_books(), 99, "철수")


def test_lend_blank_borrower_raises():
    with pytest.raises(ValueError):
        lend_book(make_books(), 1, "   ")


def test_return_book_marks_available():
    books = make_books()
    lend_book(books, 1, "철수")
    book = return_book(books, 1)
    assert book["status"] == "available"


def test_return_available_book_raises():
    # 대출하지 않은 책은 반납할 수 없다.
    with pytest.raises(ValueError):
        return_book(make_books(), 1)


def test_return_missing_book_raises():
    with pytest.raises(ValueError):
        return_book(make_books(), 99)
```

A-8. 대출과 반납 테스트 2차 (5장)

```
"""
test_lending_v2.py - 대출과 반납 약속 (2단계 5장, 2차 버전: 대출 기간 추가)

5장 요구사항 변경: "대출하면 대출 기간 14일(loan_days)이 함께 기록된다.
반납하면 대출 기간 기록은 지운다."

이 파일은 사람이 승인한 뒤에만 test_lending.py 대신 쓴다.
AI 는 이 파일을 고칠 수 없다(권한 설정으로 막는다).
"""

import pytest

from library import add_book, lend_book, return_book


def make_books():
    books = []
    add_book(books, "데미안", "헤세")
    return books


def test_lend_book_marks_lent_with_loan_days():
    books = make_books()
    book = lend_book(books, 1, "철수")
    assert book == {
        "id": 1, "title": "데미안", "author": "헤세",
        "status": "lent", "borrower": "철수", "loan_days": 14,
    }


def test_return_book_removes_loan_days():
    books = make_books()
    lend_book(books, 1, "철수")
    book = return_book(books, 1)
    assert book == {"id": 1, "title": "데미안", "author": "헤세",
                    "status": "available", "borrower": None}


def test_lend_lent_book_raises():
    books = make_books()
    lend_book(books, 1, "철수")
    with pytest.raises(ValueError):
        lend_book(books, 1, "영희")


def test_lend_missing_book_raises():
    with pytest.raises(ValueError):
        lend_book(make_books(), 99, "철수")


def test_lend_blank_borrower_raises():
    with pytest.raises(ValueError):
        lend_book(make_books(), 1, "   ")


def test_return_available_book_raises():
    with pytest.raises(ValueError):
        return_book(make_books(), 1)


def test_return_missing_book_raises():
    with pytest.raises(ValueError):
```

```
        return_book(make_books(), 99)
```

A-9. 4장 연습용 초안 (검증 전)

```
"""
library.py - 도서 대출 목록 (AI 가 만든 초안, 아직 검증 전)

4장에서 이 파일을 프로젝트 폴더의 library.py 로 복사해서 검증합니다.
"""

import json
from pathlib import Path


def add_book(books, title, author):
    """책을 등록한다. 번호는 1, 2, 3 ... 순서대로 붙는다."""
    new_id = max((b["id"] for b in books), default=0) + 1
    book = {"id": new_id, "title": title.strip(), "author": author.strip(),
            "status": "available", "borrower": None}
    books.append(book)
    return book


def list_books(books, status=None):
    """책 목록을 돌려준다. status 를 주면 그 상태의 책만 돌려준다."""
    if status is None:
        return books
    return [b for b in books if b["status"] == status]


def _find(books, book_id):
    """번호로 책을 찾는다. 없으면 ValueError 를 일으킨다."""
    for b in books:
        if b["id"] == book_id:
            return b
    raise ValueError("책을 찾을 수 없습니다.")


def lend_book(books, book_id, borrower):
    """책을 대출한다. 빌리는 사람 이름을 기록한다."""
    book = _find(books, book_id)
    if not borrower.strip():
        raise ValueError("빌리는 사람 이름을 입력하세요.")
    book["status"] = "lent"
    book["borrower"] = borrower.strip()
    return book


def return_book(books, book_id):
    """책을 반납한다."""
    book = _find(books, book_id)
    if book["status"] != "lent":
        raise ValueError("대출 중이 아닌 책입니다.")
    book["status"] = "available"
    return book


def save_books(books, path):
    """책 목록을 JSON 파일로 저장한다."""
    Path(path).write_text(json.dumps(books, ensure_ascii=False, indent=2), encoding="utf-8")


def load_books(path):
    """JSON 파일을 읽는다. 파일이 없으면 빈 목록을 돌려준다."""
    p = Path(path)
    if not p.exists():
```

```
        return []
    return json.loads(p.read_text(encoding="utf-8"))
```